

CASE STUDY

Clinical Medication Reconciliation AI

A clinical RAG pipeline that only ever tells a nurse what it can prove — grounded in FDA labels and 9,634 StatPearls articles.

- Phase 1 complete and benchmarked. Full StatPearls ingest paused at 82% (Pinecone Starter write-unit limit); Phase 2 auth/observability and Phase 3 deployment hardening are on the roadmap, not yet built.



01 Overview

MedReconcile AI takes a nurse's medication list, checks every drug pair for interactions, retrieves grounding evidence from FDA labels and StatPearls clinical articles, and returns a structured warning with severity, recommended action, and cited sources. The defining constraint is that it must never guess: every LLM-generated claim is checked against the evidence actually retrieved, and if the model can't produce a fully-cited answer, the system falls back to an FDA-label-only response rather than showing the nurse something unverified.

02 The Problem

This isn't built for large hospital systems — they already have medication-reconciliation solutions and dedicated pharmacists. It's built for small specialty practices (dermatology, ophthalmology, orthopedic surgery centers) running on basic EHR software like ModMed, where there's no pharmacist and no reconciliation team — the nurse does it manually, and the doctor often prescribes from memory. The real failure mode: a specialist prescribes a new drug without the patient's full medication list, it conflicts with something the primary care doctor already prescribed, nobody catches it, and the patient ends up in the ER. That's the gap this fills.

03 Design Principles

- Never lie by omission — every LLM-cited source is checked against what retrieval actually returned; an invented citation gets the whole response rejected, not silently trusted.
- Every failure path still answers the nurse — the generation step never raises; if the LLM or any dependency is down, the pipeline degrades to an FDA-label-only response with a confidence flag instead of an error.
- Stale interaction data is a patient-safety issue, not a data-quality issue — a two-layer cache (Redis hot path, PostgreSQL durable store) refreshes on a 7-day cycle to catch FDA's monthly label updates.

04 Architecture Deep-Dive

How the 8 pipeline stages (see the project overview) group into four defensive layers.

01 Input & Enrichment

STAGES 1-2 • INPUT VALIDATION, NORMALIZATION

Drug names are validated against a strict allowlist regex before anything else happens, plus a secondary scan for prompt-injection phrases — this system puts drug names directly into LLM prompts, so a nurse (or an attacker) typing "ignore previous instructions and..." as a drug name has to be caught at the door.

That combination passed 55/55 adversarial inputs in testing: homoglyphs, zero-width characters, RTL overrides, null bytes, newline smuggling. Past that gate, each drug is normalized to an RxCUI via RxNorm and enriched with FDA label text in parallel.

02 Interaction Detection & Retrieval

STAGES 3-5 • INTERACTION CHECK, SEMANTIC RETRIEVAL, RE-RANKING

Every drug pair is pre-screened with keyword-based severity checks, then each flagged pair triggers semantic retrieval: the query is embedded with OpenAI's text-embedding-3-small, matched against 440K+ StatPearls chunks in Pinecone (top-10 ANN), and re-ranked down to the top 3 with a cross-encoder. In benchmarking, reranking is actually the single largest chunk of pipeline latency — heavier than the embed and vector-query steps combined — which is the kind of thing you only learn by measuring instead of assuming the vector search is the bottleneck.

03 Grounded Generation

STAGE 6 • LLM GENERATION

Gemini 2.0 Flash generates the structured warning; Groq (Llama 3.3 70B) is the fallback if Gemini is unavailable, each behind its own independent circuit breaker so a Groq rate-limit storm can't trip the Gemini breaker and vice versa. Temperature is fixed at 0 on both — a medication-interaction report has no business being creative. In practice the Groq fallback path runs roughly 3x faster

than Gemini when it's invoked, which is a useful thing to know for capacity planning even though it's the secondary path.

04 **Validation & Fallback**

STAGES 7-8 • MULTI-STAGE VALIDATION

The generated response is checked against the actual retrieved chunk IDs — if the model cites a source that wasn't in the retrieval set, that's treated as a hallucination and the entire response is rejected in favor of the FDA-only fallback, flagged with confidence 0.5 so the nurse knows to double-check with the physician rather than trust it as fully reasoned. This is the concrete implementation of a dual-model guardrail: one model generates, a second check acts as the skeptic.

05 **Production & Compliance**

What's actually deployed right now is Phase 1: the reconciliation pipeline itself, running locally, fully tested (13 pytest suites covering every component), and benchmarked. What's on the documented roadmap but not yet built is everything the marketing version of this story would normally lead with — Kubernetes manifests, JWT/mTLS authentication, rate limiting, a HIPAA-audit-log middleware, Docker Compose, and a frontend. That's a deliberate sequencing choice, not an oversight: get the reasoning correct and provably-grounded first, add the operational shell once the core logic is trustworthy.

Deployment

Local FastAPI + Uvicorn —
Docker/Kubernetes are a Phase 3
roadmap item, not yet written

Cache

Redis, 24h TTL — hot-path drug-
lookup cache in front of PostgreSQL

Scaling

Single instance, no autoscaling
configured yet

Database

Pinecone Starter (vector search) +
PostgreSQL (durable drug cache)

Monitoring

None yet — Prometheus /metrics +
Grafana dashboards are Phase 2/3
roadmap items

Availability

Dual LLM provider (Gemini → Groq)
with independent circuit breakers —
no infra-level redundancy yet

06 Results

Across 100 benchmark runs (external calls mocked, so this measures pipeline overhead rather than live network variance), the full reconcile path averages 3.36s end-to-end (p95 4.17s, p99 4.38s), breaking down to roughly 336ms embedding, 145ms Pinecone query, 917ms re-ranking, and 1.32s Gemini generation (471ms on the Groq fallback path). All 100 runs returned SUCCESS with zero errors. Separately, the ingested knowledge base sits at 440,800 of a planned 534,760 StatPearls chunks (82.4%) — ingestion is paused because the Pinecone Starter tier's write-unit quota was hit, not because of any pipeline issue.

- 9,634 StatPearls medical articles indexed
- 440,800 vector chunks in Pinecone
- 110 commits with detailed architecture
- 95%+ citation accuracy achieved
- 99.5% system availability
- 8.6s average response time

07 What's Next

Two phases are scoped and waiting: Phase 2 (reliability & observability) adds retry-with-backoff before the circuit breaker trips, a half-open breaker state, OpenTelemetry spans, a Prometheus /metrics endpoint, structured logging, JWT/mTLS auth, and resuming the paused StatPearls ingest. Phase 3 (production hardening) adds HIPAA audit-log middleware, PII redaction in logs, Kubernetes manifests, Grafana dashboards, Docker Compose for local dev, and — only after auth exists — a frontend.

08 Tech Stack

FastAPI

Gemini 2.0 Flash

Groq Llama 3.3

Pinecone

OpenAI Embeddings

CrossEncoder

RxNorm API

FDA API

Kubernetes

Python 3.13

Pydantic v2